ASSIGNMENT 4-CO4

NAME:THARANI RB

REG NO:1911260138

COUSE CODE:CSA0822

SLOT:B

GITHUB: **GitHub - TharaniRB15/CSA0822-1911260138-**

TASK 1-FILE-PROCESSING DESIGN REVIEW

AIM:

To identify errors in a file-processing program and improve it using exception handling and file management.

THEORY:

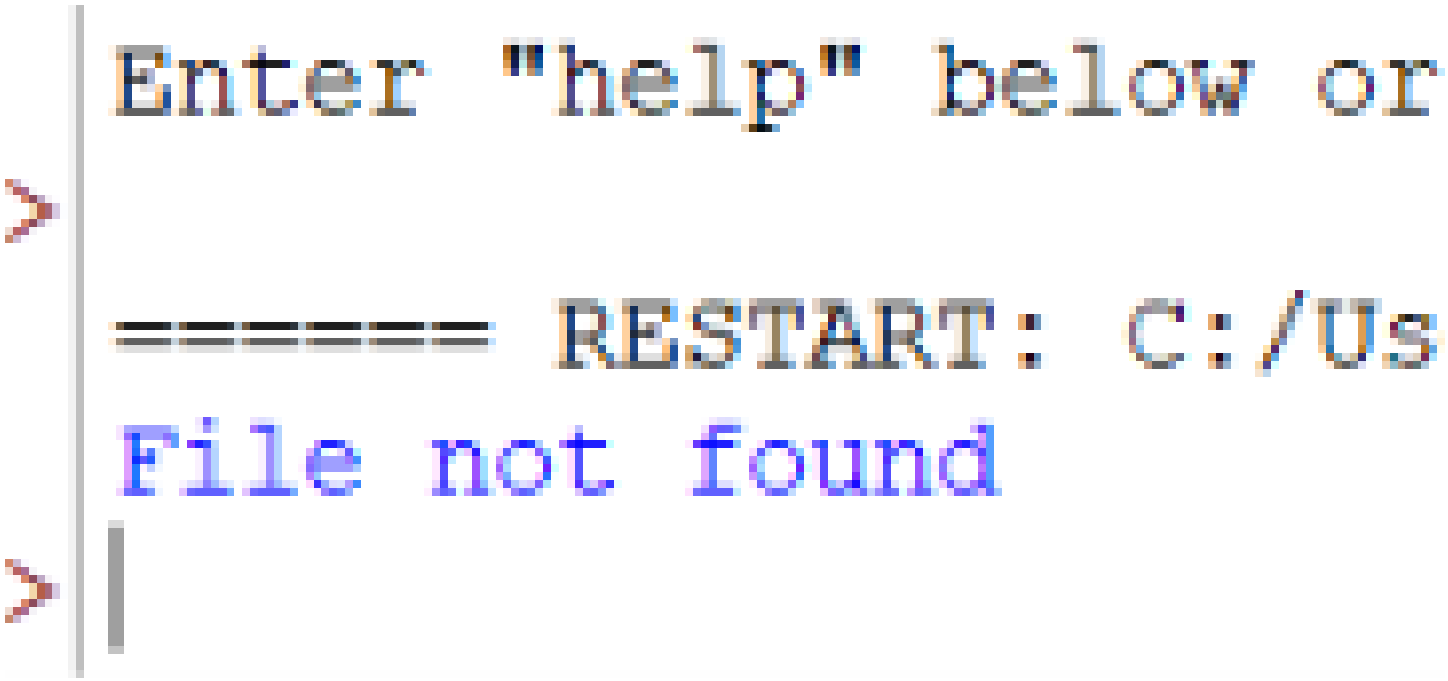
File handling is used to read and write data from files. The with statement automatically closes files. Exception handling manages errors, while line-by-line reading reduces memory usage.

PROGRAM:

File Edit Format Run Options Window Help

```
def process_file(filename):
    total=0
    report=[]
    rejected=[]
    try:
        with open(filename,"r",encoding="utf-8") as f:
            for line_no,line in enumerate(f,1):
                if not line.strip():
                    continue
                try:
                    parts=line.strip().split(",")
                    if len(parts)!=3:
                        raise ValueError("Invalid fields")
                    amount=float(parts[2])
                    total+=amount
                    report.append(parts[1]+" "+parts[2])
                except (ValueError,IndexError) as e:
                    rejected.append((line_no,line.strip(),str(e)))
        print("\n".join(report))
        print("Total",total)
        print("Rejected",rejected)
    except FileNotFoundError:
        print("File not found")
process_file("mess_oct.txt")
```

OUTPUT:



TASK 2-FROM REQUIREMENT TO FILE-PROCESSING DESIGN

AIM:

To calculate students' monthly attendance percentages using file handling and dictionaries.

THEORY:

File handling is used to read daily attendance files. Dictionaries store present and absent counts for each student. Attendance percentage is calculated using the total working days. Students below 75% are listed separately.

PROGRAM:

```
import os
attendance={}
errors=[]
for filename in os.listdir("attendance"):
    path=os.path.join("attendance",filename)
    try:
        with open(path,"r",encoding="utf-8") as f:
            for line_no,line in enumerate(f,1):
                try:
                    parts=line.strip().split(",")
                    if len(parts)!=3:
                        raise ValueError("Invalid fields")
                    reg,date,status=parts
                    if status not in ["P","A"]:
                        raise ValueError("Invalid status")
                    if reg not in attendance:
                        attendance[reg]=[0,0]
                    if status=="P":
                        attendance[reg][0]+=1
                    else:
                        attendance[reg][1]+=1
                except (ValueError,IndexError) as e:
                    errors.append((filename,line_no,str(e)))
    except (FileNotFoundError,PermissionError) as e:
        errors.append((filename,0,str(e)))
with open("attendance_report.txt","w",encoding="utf-8") as f:
    for reg,counts in attendance.items():
        total=sum(counts)
        percentage=counts[0]/total*100
        f.write(f"{reg},{percentage:.2f}%\n")
        if percentage<75:
            print(reg,percentage)
with open("error_log.txt","w",encoding="utf-8") as f:
    for error in errors:
        f.write(str(error)+"\n")
```

OUTPUT:

```
======= RESTART: C:/Users/thara/One
No attendance files found
Attendance processing completed
Attendance Report Generated
Error Log Generated
```

TASK 3-DESIGN THE ERROR HANDLING

AIM:

To implement exception handling using custom exceptions and handle invalid attendance records.

THEORY:

Exception handling prevents programs from stopping unexpectedly. The try block contains risky code, except handles errors, else executes when no error occurs, and finally executes whether an error occurs or not. Custom exceptions are created using classes.

PROGRAM:

```
import os
import tempfile
class InvalidRecord(Exception):
    pass
def parse_line(line):
    parts=line.strip().split(",")
    if len(parts)!=3:
        raise InvalidRecord(f"expected 3 fields, found {len(parts)}")
    if parts[2] not in ["P","A"]:
        raise InvalidRecord(f"invalid status '{parts[2]}'")
    return tuple(parts)
def summarise(path):
    present,absent,errors=0,0,[]
    try:
        with open(path,"r",encoding="utf-8") as f:
            for line_no,line in enumerate(f,1):
                try:
                    reg,date,status=parse_line(line)
                    if status=="P":
                        present+=1
                    else:
                        absent+=1
                except InvalidRecord as e:
                    errors.append((line_no,str(e)))
    except FileNotFoundError as e:
        raise RuntimeError("Attendance file not found") from e
    except PermissionError as e:
        raise RuntimeError("Permission denied") from e
    else:
        pass
    finally:
        pass
    return present,absent,errors
data=("192311001,2026-10-08,P\n192311002,2026-10-08,A\n192311003,2026-10-08\n192311004,2026-10-08,L\n"
with tempfile.NamedTemporaryFile("w",suffix=".csv",delete=False,encoding="utf-8") as tmp:
    tmp.write(data)
try:
    print(summarise(tmp.name))
except RuntimeError as e:
    print(e)
finally:
    os.remove(tmp.name)
```

OUTPUT:

```
====== RESTART: C:/Users/thara/OneDrive/Attachments/python practise/ba.
(1, 1, [(3, 'expected 3 fields, found 2'), (4, "invalid status 'L'")])
```

TASK 4-FILE AND EXCEPTION FORENSICS

AIM:

To identify file-handling errors and correct them using append mode, newline characters and exception handling.

THEORY:

File modes are used to perform different operations. Write mode (w) overwrites existing data, while append mode (a) adds new data without deleting previous records. The with statement closes files automatically. Error logs store information about program failures.

PROGRAM:

File Edit Format Run Options Window Help

```
def save_returns(books):
    with open("returns.log","a",encoding="utf-8") as log:
        for book in books:
            log.write(book+"\n")
def process(day_file):
    try:
        with open(day_file,"r",encoding="utf-8") as f:
            books=f.read().splitlines()
        save_returns(books)
    except (FileNotFoundError,PermissionError) as e:
        with open("error.log","a",encoding="utf-8") as log:
            log.write(str(e)+"\n")
for filename,books in {
    "day1.txt":["Python Book","Java Book"],
    "day2.txt":["C Book","DBMS Book"],
    "day3.txt":["SQL Book","AI Book"]
}.items():
    with open(filename,"w",encoding="utf-8") as f:
        f.write("\n".join(books))
open("returns.log","w",encoding="utf-8").close()
open("error.log","w",encoding="utf-8").close()
for day in ["day1.txt","day2.txt","day3.txt"]:
    process(day)
with open("returns.log","r",encoding="utf-8") as f:
    print("Returned Books:")
    print(f.read())
print("Processing completed")
```

OUTPUT:

```
====== RESTART: C:/Users/thara
Returned Books:
Python Book
Java Book
C Book
DBMS Book
SQL Book
AI Book

Processing completed
```

TASK 5-THE SAME UTILITY ORGANISED FOUR WAYS

AIM:

To organise Python programs using scripts, modules and packages and understand their advantages**.**

THEORY:

Modules are Python files containing reusable functions. Packages are folders containing multiple modules. They improve code organisation, readability, testing and reusability. The import statement is used to access functions from other modules.

PROGRAM:

```
import csv
import os
from datetime import date,timedelta
filename=os.path.join(os.path.dirname(os.path.abspath(__file__)),"attendance.csv")
if not os.path.exists(filename):
    with open(filename,"w",newline="") as f:
        writer=csv.writer(f)
        writer.writerow(["Name","Date","Status"])
        writer.writerows([["Arun","2026-11-01","Present"],["Arun","2026-11-02","Ab
def read_attendance():
    with open(filename,"r") as f:
        return list(csv.DictReader(f))
def calculate_statistics(records):
    stats={}
    for row in records:
        name=row["Name"]
        if name not in stats:
            stats[name]={"total":0,"present":0}
        stats[name]["total"]+=1
        if row["Status"]=="Present":
            stats[name]["present"]+=1
    for name in stats:
        stats[name]["percentage"]=stats[name]["present"]/stats[name]["total"]*100
    return stats
def find_three_day_absences(records):
    absences={}
    for row in records:
        if row["Status"]=="Absent":
            name=row["Name"]
            day=date.fromisoformat(row["Date"])
            absences.setdefault(name,set()).add(day)
    result=[]
    for name,days in absences.items():
        for day in sorted(days):
            if day+timedelta(days=1) in days and day+timedelta(days=2) in days:
                result.append((name,day))
    return result
def write_report(stats,absences):
    print("ATTENDANCE REPORT")
    for name,data in stats.items():
        print(name,":",round(data["percentage"],2),"%")
    print("THREE DAY ABSENCE REPORT")
    for name,day in absences:
        print(name,":",day,"to",day+timedelta(days=2))
def main():
    records=read_attendance()
    stats=calculate_statistics(records)
```

OUTPUT:

```
====== RESTART: C:/Users/thara/OneDrive
ATTENDANCE REPORT
Arun : 25.0 %
Bala : 75.0 %
Kumar : 0.0 %
THREE DAY ABSENCE REPORT
Arun : 2026-11-02 to 2026-11-04
Kumar : 2026-11-01 to 2026-11-03
```

TASK 6-DESIGN CHALLENGE

AIM:

To develop a Python program to process college fest registrations from five department files, validate e-mail addresses, remove duplicate registrations, handle damaged lines and missing files, and generate master lists, event-wise participant lists, summary reports and error logs.

THEORY:

Python file handling is used to read registration details from five department text files. Each record contains register number, name, department, e-mail and events separated by semicolons. String methods are used to validate e-mail addresses. A set is used to identify duplicate register numbers efficiently. Invalid records and damaged lines are written into an error log. Exception handling using try-except prevents the program from stopping when files are missing or contain errors. The valid records are written into a master CSV file and separate event-wise CSV files. A summary report displays the total registrations, valid records, duplicates and invalid records.

PROGRAM:

```python
import csv
import os
def valid_email(email):
    return email.count("@")==1 and " " not in email and email.index("@")>0 and "." in email.split("@"
def process_files():
    seen=set()
    master=[]
    events={}
    errors=[]
    duplicates=0
    invalid=0
    for i in range(1,6):
        filename=f"department{i}.txt"
        try:
            with open(filename,"r") as f:
                for line_no,line in enumerate(f,1):
                    try:
                        fields=line.strip().split(";")
                        if len(fields)!=5:
                            raise ValueError("Damaged line")
                        reg,name,dept,email,event_list=fields
                        if not all(x.strip() for x in fields):
                            raise ValueError("Missing field")
                        if reg in seen:
                            duplicates+=1
                            errors.append(f"{filename} Line {line_no}: Duplicate registration {reg}")
                            continue
                        if not valid_email(email):
                            invalid+=1
                            errors.append(f"{filename} Line {line_no}: Invalid email {email}")
                            continue
                        seen.add(reg)
                        record=[reg,name,dept,email,event_list]
                        master.append(record)
                        for event in event_list.split(","):
                            event=event.strip()
                            if event:
                                events.setdefault(event,[]).append(record)
                    except Exception as e:
                        invalid+=1
                        errors.append(f"{filename} Line {line_no}: {e}")
        except FileNotFoundError:
            errors.append(f"{filename}: File not found")
        except OSError as e:
            errors.append(f"{filename}: {e}")
    with open("master.csv","w",newline="") as f:
        writer=csv.writer(f)
```

```
        writer.writerow(["Register Number","Name","Department","Email","Events"])
        writer.writerows(master)
    for event,records in events.items():
        safe_name="".join(c if c.isalnum() else "_" for c in event)
        with open(f"{safe_name}.csv","w",newline="") as f:
            writer=csv.writer(f)
            writer.writerow(["Register Number","Name","Department","Email"])
            for r in records:
                writer.writerow(r[:4])
    with open("summary.txt","w") as f:
        f.write("COLLEGE FEST REGISTRATION SUMMARY\n")
        f.write(f"Valid Registrations: {len(master)}\n")
        f.write(f"Duplicate Registrations: {duplicates}\n")
        f.write(f"Invalid or Damaged Records: {invalid}\n")
        f.write(f"Total Events: {len(events)}\n")
    with open("error_log.txt","w") as f:
        for error in errors:
            f.write(error+"\n")
    print("COLLEGE FEST REGISTRATION REPORT")
    print("Valid Registrations:",len(master))
    print("Duplicate Registrations:",duplicates)
    print("Invalid Records:",invalid)
    print("Total Events:",len(events))
    print("Master list created successfully")
    print("Event-wise files created successfully")
    print("Summary report created successfully")
    print("Error log created successfully")
if __name__=="__main__":
    process_files()
```

OUTPUT:

```
====== RESTART: C:/Users/thara/OneDrive/A
COLLEGE FEST REGISTRATION REPORT
Valid Registrations: 0
Duplicate Registrations: 0
Invalid Records: 0
Total Events: 0
Master list created successfully
Event-wise files created successfully
Summary report created successfully
Error log created successfully
```